UNIVERSITY OF ATHENS
DEPARTMENT OF INFORMATICS AND TELECOMMUNICATIONS

Deep Learning for NLP

Student name: *Georgios Fragkakis*
sdi2100206

Course: *Artificial Intelligence II (M138, M226, M262, M325)*
Semester: *Spring Semester 2025*

Contents

1 Abstract **2**

2 Data processing and analysis **2**
2.1 Pre-processing . . . 2
2.2 Data partitioning for train, test and validation . . . 2
2.3 Vectorization . . . 3

3 Algorithms and Experiments **3**
3.1 Experiments . . . 3
3.2 Evaluation . . . 4
3.2.1 ROC curve . . . 4
3.2.2 Learning Curve . . . 5
3.2.3 Confusion matrix . . . 6

4 Results and Overall Analysis **7**
4.1 Results Analysis . . . 7
4.2 Comparison with the first project . . . 7

5 Bibliography **8**

1. Abstract

This project implements a Twitter sentiment classifier using a Deep Neural Network with averaged word vector representations. Through comprehensive preprocessing of social media text and optimized network training, we aim for high validation accuracy and F1-score. Key elements include leveraging pre-trained embeddings, applying network normalization and dropout for regularization, and ensuring reproducibility.

2. Data processing and analysis

2.1. Pre-processing

To make the Twitter data usable for sentiment classification, a crucial preprocessing step is necessary. The following techniques were implemented to try to clean and preprocess the text data for my model

- **Repeated character normalization:** Reduces variations in spelling and noise, improving the model's ability to recognize words and their sentiment.
  (example: "soooo" -> "so")
- **URL removal:** Removes URLs as they rarely contribute to sentiment analysis and can introduce noise.
  (example: "www.fragkakis.com" -> "URL")
- **Extra white space removal:** Simple but effective method.
  (example: "AI II" -> "AI II")
- **Twitter Elements**:
  - **Hashtag Removal:** Simply removing the hashtags had the best impact improving model accuracy , in comparison with any other hashtag preprocessing i tried implementing.
    (example: "#GreatDay" $\rightarrow$ "GreatDay")
  - **User Mention Removal:** Eliminates irrelevant user names, focusing the model on the actual content of the tweet.
    (example: "@giorgos" -> "MENTION")

2.2. Data partitioning for train, test and validation

The experiment used the below pre-defined datasets for the assignment:

- **Training Dataset:** Used for model training and cross-validation.
- **Validation Dataset:** Used for the evaluation of the model trained on the whole training dataset.
- **Test Dataset:** Reserved exclusively for producing the output file submission.csv for the kaggle competition.

2.3. Vectorization

To represent the tweets numerically for the deep learning model, we employed vectorization based on pre-trained word embeddings. We used the 200-dimensional GloVe vectors trained on Twitter data (glove.twitter.27B.200d), chosen for their ability to capture semantic relationships between words relevant to the dataset.

After text preprocessing, each tweet was converted into a single fixed-length vector by averaging the GloVe vectors corresponding to the words present in that tweet.This resulting 200-dimensional averaged vector served as the input features for our neural network, providing a dense semantic representation of the tweet's content.

3. Algorithms and Experiments

Trial	Preprocessing	GloVe File	Model + Parameters	Accuracy
1st Model	None	glove.6B.50d.txt	2 layers + Adam	0.6989
2nd Model	Stopwords + lemmitization	glove.twitter.27B.100d.txt	2 layers + AdamW	0.7429
3rd Model	Simple + Web specific data cleaning	glove.twitter.27B.200d.txt	3 layers + Adam	0.78876

Table 1: Performance evolution and Experimental trials

3.1. Experiments

- **Model 1 (Baseline):** First try to try to understand the mechanics of the assignement , while having only 69% accuracy:
  - Utilized standard, general-purpose GloVe embeddings (glove.6B.50d.txt) not specifically tailored to any domain.
  - No data preprocessing was applied, feeding raw text directly into the model
  - Employed a basic 2-layer network architecture.
- **Model 2 (Basic Optimazation):** Trying to achieve a better accuracy thru testing basic changes and while achieving 0.772 accuracy:
  - Switched to GloVe embeddings trained on a more relevant corpus (Twitter: glove.twitter.27B.100d.txt), which skyrocketed the accuracy.
  - Basic preprocessing steps, specifically Stopwords removal and Lemmatization(thought they helped at the time), were introduced to clean the text data.
  - Maintained a 2-layer network architecture but changed the optimizer to AdamW.

- **Model 3 (Specific Tuning):** Excesive optimization delivered 0.789 accuracy through:
  - Switched from twitter 100d to 200d , this made the accuracy increase
  - Made the pre proccessing more basic ,by removing the stopwords removal and Lemmatization, but introduced twitter specific data cleaning (url , mentions) , which also helped the accuracy go up a lot
  - Increased the model's capacity by adding a layer, resulting in a 3-layer architecture.
  - ReduceLROnPlateau scheduler really helped
  - Early stopping also helped prevent the model from overfitting and reduced training time in general
  - Switched back to Adam optimizer (also boosted accuracy).
  - Lowering batch size to under 100 gave a good boost in accuracy.

3.2. Evaluation

Final model validation score:

```
Epoch 8/30 | Train Loss: 0.4385 | Train Acc: 0.7926 | Val Loss: 0.4447 | Val Acc: 0.7890 | Val F1: 0.7841
*** New best model with Val Acc: 0.7890 ***
```

Figure 1: Score of the final model

3.2.1. ROC curve.

An ROC (Receiver Operating Characteristic) curve is a graph showing the performance of a binary classification model at different decision thresholds. It plots the True Positive Rate (sensitivity) against the False Positive Rate (1-specificity). The curve helps visualize how well the model can distinguish between the positive and negative classes and the trade-off between identifying true positives and avoiding false positives.

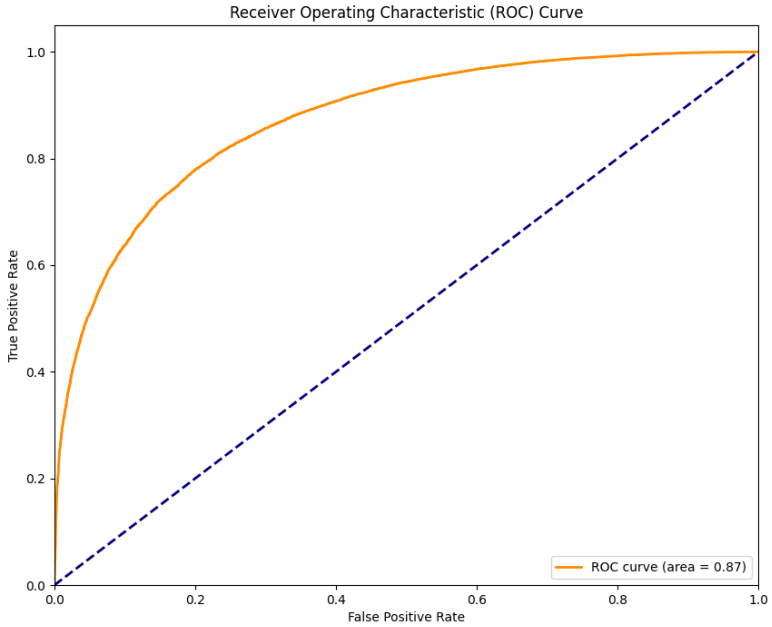


Figure 2: ROC Curve

3.2.2. Learning Curve.

The training and validation loss curves show a general downward trend over the course of 10+ epochs. The training loss decreases steadily throughout, while the validation loss initially decreases and then levels off with slight fluctuations in the later epochs. In the accuracy plot, both training and validation accuracy increase over time. Training accuracy consistently rises and remains higher than validation accuracy. Validation accuracy also shows a gradual increase, with minor variations as training progresses.

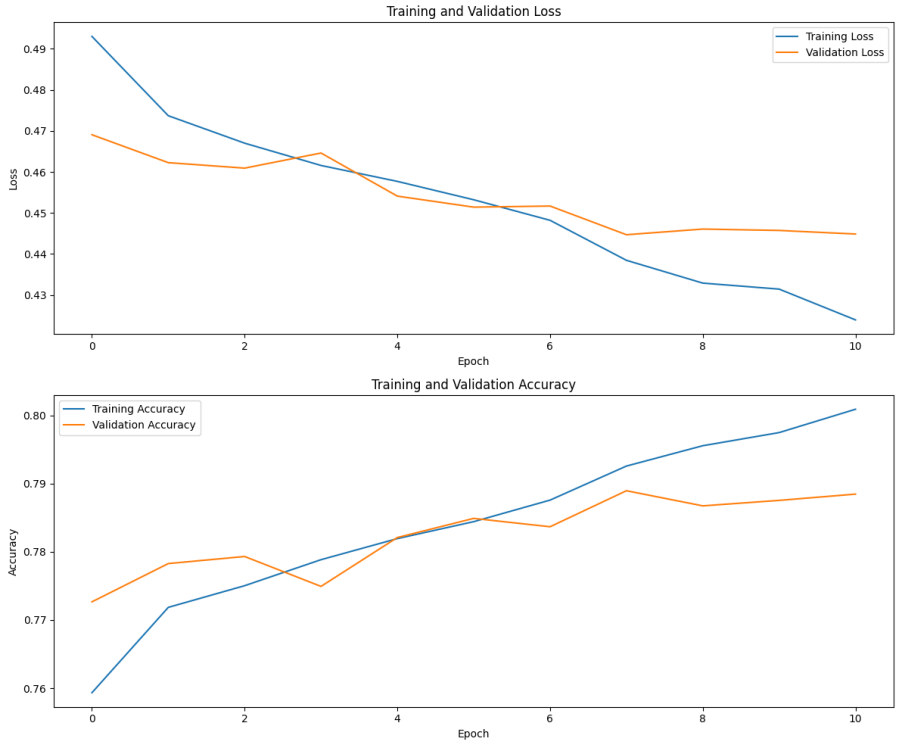


Figure 3: Learning Curves for this model

3.2.3. Confusion matrix.

Figure 4 presents the confusion matrix. The model correctly classified 16,543 negative instances (True Negatives) and 16,885 positive instances (True Positives), while misclassifying 4,654 negative instances as positive (False Positives) and 4,314 positive instances as negative (False Negatives).

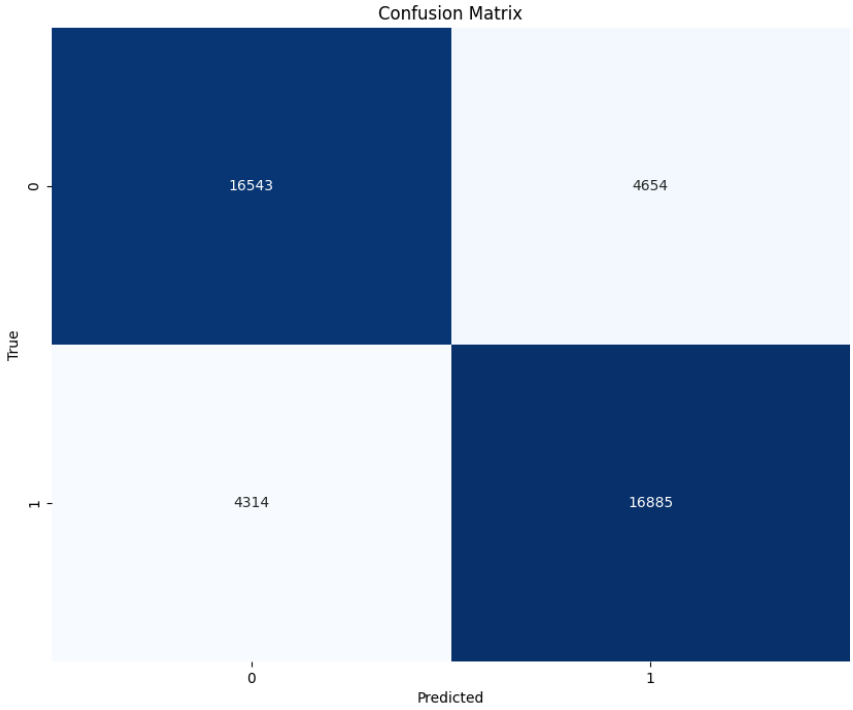


Figure 4: Confusion Matrix on Validation dataset

4. Results and Overall Analysis

4.1. Results Analysis

- Achieved 0.7890 accuracy on the validation set
- Achieved 0.7882 accuracy on the test set
- Effective on direct sentiment expressions
- Best balance between validation and test dataset acurracy

4.2. Comparison with the first project

The results from the first project (Logistic Regression with TF-IDF) slightly outperformed this neural network approach, achieving 80.25% validation accuracy compared to 78.82%. This performance difference can be attributed to several factors: the TF-IDF n-gram representation likely captured important phrase-level patterns that averaged word embeddings couldn't preserve, more extensive preprocessing in the first project optimized the input data, and the carefully tuned logistic regression hyperparameters (liblinear solver, C=1.2, L2 regularization) proved particularly effective for this task.The neural network approach suffered primarily from information loss through

embedding averaging, which eliminated word order and sequential information critical to sentiment analysis. Additionally, while Twitter-specific GloVe embeddings were used, they may not have perfectly captured the sentiment nuances present in this specific dataset. This comparison demonstrates that more complex models don't always give you better results - traditional methods with carefully engineered features can sometimes outperform neural approaches.

5. Bibliography

References

[1] *Introduction to PyTorch*.
URL: https://pytorch.org/tutorials/beginner/basics/intro.html

[2] *GloVe: Global Vectors for Word Representation*.
URL: https://nlp.stanford.edu/projects/glove/